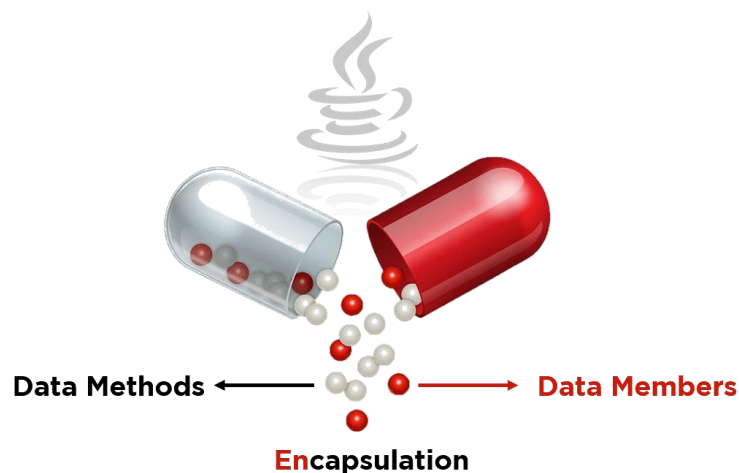


ENCAPSULATION IN JAVA

Encapsulation in Java refers to integrating data (**variables**) and code (methods) into a single unit. In encapsulation, a class's variables are hidden from other classes and can only be accessed by the methods of the class in which they are found.



Example of Data Encapsulation

Below is a program to calculate the perimeter of a rectangle:

```
class Perimeter {  
    int length;  
    int breadth;  
    Perimeter(int length, int breadth) {  
        this.length = length;  
        this.breadth = breadth;  
    }  
    public void getPerimeter() {  
        int perimeter = 2 * (length + breadth);  
        System.out.println("Perimeter of Rectangle : " + perimeter);  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        Perimeter rectangle = new Perimeter(3, 6);  
        rectangle.getPerimeter();  
    }  
}
```

Output:

```
Perimeter of Rectangle : 18
```

❖ Data Hiding

Access modifiers are used to achieve data hiding. Access modifiers are the keywords that specify the accessibility or the scope of methods, classes, fields, and other members.

Difference between encapsulation and data hiding is that Encapsulation is a way of bundling data whereas Data Hiding prevents external unauthorized access to the sensitive data.

The **four** types of access modifiers in Java are:

- **Public:** A public modifier is accessible to everyone. It can be accessed from within the class, outside the class, within the package, and outside the package.

```
class A {  
  
    // public variables  
    public int a;  
    public int b;  
  
    // public methods  
    public void method_1() {  
        // Method Logic  
    }  
}
```

- **Private:** A private modifier can not be accessed outside the class. It provides restricted access. Example:

```
class A {  
  
    // private variables  
    private String a;  
    private int b;  
}
```

- **Protected:** A protected modifier can be accessed from the classes of the same package and outside the package only through inheritance.

```
class A {
    // protected variables
    protected String a;
    protected int b;
}
```

- **Default:** A default modifier is accessible only within the package. If no modifier is used, it is treated as default.

Access Specifier Item ▼	Default	Public	Protected	Private
Class	Yes	Yes	No	No
Inner Class	Yes	Yes	Yes	Yes
Interface	Yes	Yes	No	No
Interface Inside Class	Yes	Yes	Yes	Yes
enum	Yes	Yes	No	No
enum Inside Class	Yes	Yes	Yes	Yes
enum inside Interface	Yes	No	No	No
Constructor	Yes	Yes	Yes	Yes
methods & data inside class	Yes	Yes	Yes	Yes
methods & data inside Interface	Yes	No	No	No

Or,

Modifier	Class	Package	Subclass	Global
Public	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	No
Default	Yes	Yes	No	No
Private	Yes	No	No	No

Example of Data Hiding:

```
class Employee {

    // private variables
    private String name;
    private int age;
    private int salary;
}

public class Main {

    public static void main(String[] args) {
        //object of class Employee
        Employee newObj = new Employee();
        newObj.name = "James";
        System.out.println(newObj.name);
    }
}
```

Output:

```
Main.java:5: error: name has private access in Person
myObj.name = "James";
^
Main.java:6: error: name has private access in Person
System.out.println(myObj.name);
^
2 errors
```

❖ How to implement Encapsulation in Java?

We need to perform two steps to achieve the purpose of Encapsulation in Java.

- Use the private access modifier to declare all variables/fields of class as private.
- Define public getter and setter methods to read and modify/set the values of the abovesaid fields.

Example:

```
class Book {  
  
    //private fields  
  
    private String author;  
    private String title;  
    private int id;  
  
    //public getter and setter methods for each field  
  
    public String getAuthor() {  
        return author;  
    }  
  
    public void setAuthor(String a) {  
        this.author = a;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public void setTitle(String t) {  
        this.title = t;  
    }  
  
    public int getId() {  
        return id;  
    }  
}
```

```
public void setId(int i) {
    this.id = i;
}

public class Main {

    public static void main(String[] args) {
        Book newObj = new Book();

        newObj.setAuthor("Jane Austen");

        newObj.setTitle("Pride and Prejudice");

        newObj.setId(775);

        System.out.println(
            "Book Title: " +
            newObj.getTitle() +
            "\n" +
            "Book Author: " +
            newObj.getAuthor() +
            "\n" +
            "Book Id: " +
            newObj.getId()
        );
    }
}
```

Output:

```
Book Title: Pride and Prejudice
Book Author: Jane Austen
Book Id: 775
```

❖ Benefits of Encapsulation java

- Cleaner, more organized and less complex code.
- More flexible code as can modify a unit independently without changing any other unit.
- Makes the code more secure.
- The code can be maintained at any point without breaking the classes that use the code.